

Mohammed Aseel - Assignment 2

Repository Link: mohdaseel/assignment2da3408

Question 1 - Docker: Naive vs Multi-Stage Builds

1. Build & Image Metrics

Naive Image Name/Tag:	spam-api:naive
Naive Image Size:	2.11 GB
Multi-Stage Image Name/Tag:	spam-api:multistage
Multi-Stage Image Size:	524 MB
Percentage Size Reduction:	75.2%

$$\text{Percentage Reduction} = (\text{Naive Size} - \text{Multi-Stage Size}) / \text{Naive Size} \times 100$$

2. Evidence of Successful Execution

```
ass2@ubuntu:~/Desktop/ass2$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
c8b8c8b8a8b   spam-api:multistage   "uvicorn main:app --"    11 seconds ago   Up 36m
c8b8c8b8a8b   spam-api:naive        "uvicorn main:app --"    7 minutes ago   Up 7m
ass2@ubuntu:~/Desktop/ass2$ curl http://localhost:8000/health
{"status": "ok"}
ass2@ubuntu:~/Desktop/ass2$ curl http://localhost:8000/health
{"status": "ok"}
ass2@ubuntu:~/Desktop/ass2$ docker images | grep spam-api
spam-api:multistage
spam-api:naive
ass2@ubuntu:~/Desktop/ass2$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"text": "Win a FREE iPhone now! Click here: bit.ly/xyz123"}'
{"label": "spam", "cache_hit": false, "latency_ms": 887.33}
ass2@ubuntu:~/Desktop/ass2$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"text": "Win a FREE iPhone now! Click here: bit.ly/xyz123"}'
{"label": "spam", "cache_hit": true, "latency_ms": 13.03}
```

3. Technical Explanation of Size Reduction

The multi-stage Docker build is significantly smaller because it isolates the runtime environment from the build toolchain. In the builder stage, temporary build tools, intermediate .whl cache files, and compilers required during package installation are utilized. In the final stage, only the lean runtime base image (python:3.11-slim) and the compiled Python site-packages copied from the builder stage are retained. Heavy static build utilities and pip caches are completely excluded from the final image layers.

Question 2 - Multi-Container Orchestration with Docker Compose

1. Architecture & Inter-Service Networking

The application consists of two containerized services defined in docker-compose.yml:

- **api Service:** Built from the multi-stage Dockerfile, exposing port 8000.
- **cache Service:** Uses redis:7-alpine, exposing port 6379.

Inter-service networking is established via Docker Compose's default internal bridge network, allowing the api service to resolve the Redis cache directly using the service hostname redis (or cache).

2. Cache Performance Evidence (HIT vs MISS)

```
Container ass2-api-1 Started
ass2@ubuntu:~/Desktop/ass2$ docker compose ps
NAME                IMAGE      COMMAND                  SERVICE  CREATED        STATUS        PORTS
ass2-api-1          ass2-api   "uvicorn main:app --"    api      About a minute ago   Up 42 seconds   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
ass2-cache-1        redis:7-alpine   "docker-entrypoint.s..." cache    About a minute ago   Up About a minute   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
ass2@ubuntu:~/Desktop/ass2$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"text": "Win a FREE iPhone now! Click here: bit.ly/xyz123"}'
{"label": "spam", "cache_hit": false, "latency_ms": 887.33}
ass2@ubuntu:~/Desktop/ass2$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"text": "Win a FREE iPhone now! Click here: bit.ly/xyz123"}'
{"label": "spam", "cache_hit": true, "latency_ms": 13.03}
```

First call (Cache MISS): 887.33ms, computed via ML model & saved to Redis. Second call (Cache HIT): 13.03ms, retrieved directly from Redis.

3. Docker Compose vs Kubernetes Scope

Docker Compose addresses single-host multi-container orchestration by managing container lifecycles, shared networking, and environment configurations on a single Docker daemon via a declarative YAML file. In contrast, Kubernetes provides enterprise-scale, multi-node cluster orchestration, managing workloads across physical or virtual machine boundaries with advanced features like automated scheduling, horizontal pod autoscaling, self-healing, and rolling updates.

Question 3 - Kubernetes Indexed Job: Parallel Data Validation

1. Parallelism Choice & Resource Justification

Given a cluster constraint of 2 nodes with 2 allocatable CPUs each (4 CPUs total), allocating cpu: "1" request/limit per pod guarantees predictable execution without CPU overcommit or throttling. Therefore, parallelism: 4 was selected because:

$$\text{Concurrent Pods} = \text{Total Available CPUs} / \text{CPU per Pod} = 4 / 1 = 4$$

With completions: 8 and parallelism: 4, the 8 data validation shards are processed in two successive parallel waves of 4 pods each.

2. Real Pod Execution Evidence (kubectl get pods -o wide)

```
(base) asee@ubuntu:~/Desktop/ass2$ kubectl get pods -l job-name=shard-validator-job -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
shard-validator-job-0-xhb7h          0/1     Completed 0           102s  10.244.0.9      minikube         <none>            <none>
shard-validator-job-1-6wmc          0/1     Completed 0           101s  10.244.0.7      minikube         <none>            <none>
shard-validator-job-2-br75          0/1     Completed 0           101s  10.244.0.6      minikube         <none>            <none>
shard-validator-job-3-tklc          0/1     Completed 0           101s  10.244.0.8      minikube         <none>            <none>
shard-validator-job-4-sghfg          0/1     Completed 0           65s   10.244.0.10     minikube         <none>            <none>
shard-validator-job-5-cj9s8          0/1     Completed 0           64s   10.244.0.12     minikube         <none>            <none>
shard-validator-job-6-qdw7          0/1     Completed 0           64s   10.244.0.11     minikube         <none>            <none>
shard-validator-job-7-gq7t7          0/1     Completed 0           63s   10.244.0.13     minikube         <none>            <none>
(base) asee@ubuntu:~/Desktop/ass2$ for pod in $(kubectl get pods -l job-name=shard-validator-job -o jsonpath='{.items[*].metadata.name}'); do
  kubectl logs $pod
done
[Pod: shard-validator-job-0-xhb7h] Node: minikube | Shard 0 validation complete: 4 invalid rows found.
[Pod: shard-validator-job-1-6wmc] Node: minikube | Shard 1 validation complete: 6 invalid rows found.
[Pod: shard-validator-job-2-br75] Node: minikube | Shard 2 validation complete: 11 invalid rows found.
[Pod: shard-validator-job-3-tklc] Node: minikube | Shard 3 validation complete: 3 invalid rows found.
[Pod: shard-validator-job-4-sghfg] Node: minikube | Shard 4 validation complete: 12 invalid rows found.
[Pod: shard-validator-job-5-cj9s8] Node: minikube | Shard 5 validation complete: 6 invalid rows found.
[Pod: shard-validator-job-6-qdw7] Node: minikube | Shard 6 validation complete: 9 invalid rows found.
[Pod: shard-validator-job-7-gq7t7] Node: minikube | Shard 7 validation complete: 4 invalid rows found.
(base) asee@ubuntu:~/Desktop/ass2$
```

3. Validation Results Collected via Kubernetes API

- Shard 0: 4 invalid rows
- Shard 1: 6 invalid rows
- Shard 2: 11 invalid rows
- Shard 3: 3 invalid rows
- Shard 4: 12 invalid rows
- Shard 5: 6 invalid rows
- Shard 6: 9 invalid rows
- Shard 7: 4 invalid rows

Storage Justification: Retrieving output logs directly via the Kubernetes API (kubectl logs) avoids configuring a shared persistent volume with ReadWriteMany access mode. Minikube's default standard hostpath storage provisioner is bound to a single node and does not natively support multi-node shared write access, making log streaming via the API the cleanest and most reliable mechanism for transient batch workloads.

4. 6-CPU Cluster Re-Justification

If the cluster capacity were expanded to 3 nodes with 2 CPUs each (6 CPUs total), the parallelism value should be updated from 4 to 6. Setting parallelism: 6 allows 6 pods (each requesting 1 CPU) to execute concurrently across the 3 nodes, completing the 8-shard workload in two faster waves (6 pods in wave 1, and 2 pods in wave 2).

Question 4 - Kubernetes Deployments: Self-Healing & Rolling Updates

1. Self-Healing Verification

```
^C(base) asee@ubuntu:~/Desktop/ass2$ kubectl get pods -l app=spam-api
NAME                                READY   STATUS    RESTARTS   AGE
spam-api-deployment-7b7bf78c7c-6rfvj 1/1     Running   0           8m28s
spam-api-deployment-7b7bf78c7c-fbnfj 1/1     Running   0           8m28s
(base) asee@ubuntu:~/Desktop/ass2$ POD_NAME=$(kubectl get pods -l app=spam-api -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $POD_NAME
pod "spam-api-deployment-7b7bf78c7c-6rfvj" deleted from default namespace
(base) asee@ubuntu:~/Desktop/ass2$ kubectl get pods -l app=spam-api
NAME                                READY   STATUS    RESTARTS   AGE
spam-api-deployment-7b7bf78c7c-7nk8v 0/1     Running   0           18s
spam-api-deployment-7b7bf78c7c-fbnfj 1/1     Running   0           8m57s
(base) asee@ubuntu:~/Desktop/ass2$ # Build the v2 image
```

Self-Healing Mechanism: Self-healing is managed by the ReplicaSet controller (under the Deployment controller). It executes a continuous reconciliation loop that compares the desired state (replicas: 2) with the actual state in the cluster. When a pod is deleted, the observed count drops to 1, prompting the ReplicaSet controller to immediately schedule a replacement pod to restore the desired count of 2.

2. Rolling Update Verification

```
kubectl rollout history deployment/span-api-deployment
Waiting for deployment "span-api-deployment" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "span-api-deployment" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "span-api-deployment" rollout to finish: 2 old replicas are pending termination...
Waiting for deployment "span-api-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "span-api-deployment" successfully rolled out
deployment.apps/span-api-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
(base) asee@ubuntu:~/Desktop/ass2$ curl $(minikube service span-api-service --url)/healthz
{"status": "ok", "version": "v2"}(base) asee@ubuntu:~/Desktop/ass2$ git status
```

Deployment vs Job During Updates: If this application were deployed as a Job instead of a Deployment, modifying the application code or container image would not trigger a zero-downtime rolling update. Kubernetes Jobs are immutable, run-to-completion objects. Updating a Job requires manually deleting the existing Job resource, causing downtime, and launching a new Job, which terminates after execution rather than continuously serving HTTP traffic.

3. Workload Fit Discussion (Serving vs Batch)

A Deployment is engineered for long-running, stateless serving workloads (such as Web APIs) where continuous uptime, high availability, load-balanced traffic, self-healing, and zero-downtime updates are required. Conversely, an Indexed Job is engineered for batch processing workloads (such as data validation or ETL pipelines) that perform a finite set of tasks to completion and exit, releasing cluster computing resources once finished.